

Fundamentals of Cross-Site Scripting (XSS) Attack

IN THIS CHAPTER, WE present a comprehensive study of one of the dangerous web application vulnerabilities, i.e. Cross-Site Scripting (XSS). This chapter focuses on what is XSS, what are the different flavors of XSS attack, how the attacker can exploit this vulnerability, what are the effects of the XSS attack, and lastly we shed some light on the defensive techniques developed by the researchers to defend against the XSS attack.

3.1 OVERVIEW OF CROSS-SITE SCRIPTING (XSS) ATTACK

XSS comes under the category of code injection attack [4]. It is one of the most severe security vulnerabilities present in the web applications. In this type of attack, adversary injects the judiciously crafted malicious JavaScript code through the input parameters at the client side. It is done in order to cause harmful actions by

the web applications and accomplish the attacker's objectives like cookie stealing and session token theft or to launch other attacks [8, 20]. The origin of XSS attack is the inappropriate filtering of the input text entered at the client side, which makes an attacker to easily introduce the mischievous code into the OSN-based web pages. These malicious scripts run at the client side in the user's web browser.

3.1.1 Steps to Exploit XSS Vulnerability

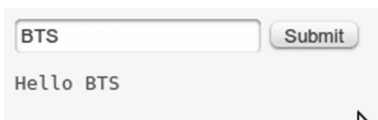
XSS arises because of the security flaws in the HTML, JavaScript, flash, AJAX, etc. When malicious code comes from a trusted source, it is executed in the same way as the legitimate JavaScript code, so the attacker is able to access the sensitive information of the victim. Here, we describe the steps to examine whether a web site is XSS vulnerable or not.

Step 1: Initially, explore the input field available in a web site. For instance, search box, comment box, or any form to be filled by the user.



Step 2: Now, enter any string into the identified field and submit it. Search for this string in the source code of the web page.

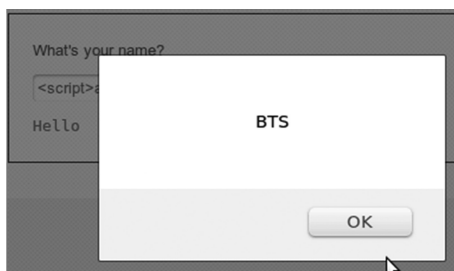
Step 3: Check if entered string is displayed on the web page, as the result of step 2.



If it is displayed then the web site may be vulnerable to XSS attack; otherwise it is not. Try for some different inputs in steps 2 and 3.

Step 4: Now enter any malicious script say, `<Script>alert("XSS");</Script>` and submit it.

Step 5: If the web page does not employ any sanitization technique, then malicious script will be executed in the browser. After its successful execution, a dialog box will pop up, reflecting the XSS attack in the message body of box.



This indicates that the web site is exposed to XSS attack. By extending the code, the attacker can steal the session token and cookie information of the user and gain access to the user's account to launch different types of attacks.

3.1.2 Recent Incidences of XSS Attack

It is a problematic task to detect XSS attack due to the relatively unchanged behavior of the browser and distinguish between illicit JavaScript from the normal web content. Almost every large online application system has been hit by the XSS worm. Web sites such as Twitter, Facebook, YouTube, and Drupal have been severely infected by the XSS attack. Table 3.1 illustrates the recent incidences of the XSS attack, along with its consequences [10, 12–14, 41].

3.2 EFFECTS OF XSS ATTACK

XSS not only enables the attackers to get their hands on the sensitive information of the user, but also enables them to trigger more

TABLE 3.1 Recent Incidences of XSS Attack

Web Applications	Year	Effects
Evernote in Windows	2018	Remote access to the victim's computer.
Trend Micro OfficeScan	2017	Sensitive information disclosure.
Cisco Prime Infrastructure	2017	Gained access to confidential browser-based information, which led to account hijacking.
Cisco ASA VPN Portal	2016	XSS attack influenced the VPN portal of Cisco, and consequently it led to credentials stealing of its users.
Drupal	2016	Account hijacking.
Ebay	2016	Hackers used parasitic code in the login page to steal the user's login details, i.e. account hijacking.
Square API	2016	The attacker injected malicious codes via login entries, which resulted in the app takeover.
NASA	2015	XSS attack vectors were detected in NASA Scientific and Technical Information (STI) Order Form, which caused disinformation to the users.
Facebook	2015	XSS bug was identified in Facebook's content delivery network, which allowed hackers to take over Facebook users' accounts.
WordPress	2015	Information disclosure.
Paypal	2015	Stored XSS vulnerability found in e-payment services permitted the hacker to insert malicious codes to launch various types of attacks.
eBay	2014	Phishing.
UK Parliament Web Site	2014	Disinformation.
RadEditor HTML Editor	2014	Improper sanitization of the user data resulted in the theft of personal information and drive-by-download attack.
Yahoo Mail	2013	The hacker utilized DOM-based XSS attack to hijack the users' account.
Internet Explorer	2013	The hacker bypassed anti-XSS filter employed in IE 8 and higher version through injecting malicious JavaScript codes into the attribute created by the attacker.

(Continued)

TABLE 3.1 (CONTINUED) Recent Incidences of XSS Attack

Web Applications	Year	Effects
eBay	2012	The attacker injected malicious codes in the product listing and caused disinformation to its users.
McAfee	2012	XSS vulnerability allowed attackers to launch drive-by-download attack.
Hotmail	2011	A security hole in Hotmail enabled attackers to steal users and cookies and take control of their session.
Amazon	2010	XSS vulnerability permitted the attacker to steal session IDs to take control over the user's account, when the user clicked on the malicious link.
Facebook	2010	XSS bug allowed hackers to hijack the user's account by posting malicious comments or posts.
Orkut	2010	Malicious group formation.
YouTube	2010	Drive-by-download.

advanced attacks using the victim's machine. Table 3.2 highlights the effects or impacts of XSS attack on the user [3].

These effects can harm the user catastrophically; therefore, web applications or software should be developed and used with proper attention, keeping the XSS flaw in mind.

3.3 CLASSIFICATION OF XSS ATTACK

There are different ways to perform XSS attack. It can be launched in three different ways and, therefore, can be classified into three categories [15]: persistent XSS attack, non-persistent XSS attack, and DOM-based XSS attack.

3.3.1 Persistent XSS Attack

It is also known as stored XSS attack because the malicious script permanently resides at the server end. In this attack, the attacker permanently injects the maliciously crafted code into the server. After this, any user who is visiting that web page with the injected script gets infected by the XSS attack. It is the most dangerous XSS attack among all types because the attacker injects the malicious

TABLE 3.2 Effects of XSS Attack

Impacts	Description
Cookie Stealing	It is possible for an attacker to steal the cookie sent by the server containing the session ID and take control of the user's account and may perform malicious activities like sending spam messages to the user's friends.
Account Hijacking	The attackers can steal the sensitive information like financial account credentials or bank account login details for the use of their benefits. If an account is hijacked, the attacker has access to the OSN server and database system and thus has complete control over the OSN web application.
Misinformation	This is a threat of credentialed misinformation. It may include malwares which may track the user's traffic statistics, leading to the loss of privacy. Moreover, these may also alter the content of the page, resulting in the loss of integrity.
Denial of Service Attack [9, 34]	Data availability is an utmost important functionality provided by any enterprise. But XSS attack can be used to redirect the user to some other fake web page so that he can't access the legitimate web site, whenever the user makes a request to that web page. Thus the attacker successfully launches the DDoS attack. Malicious scripts may also crash the user browser by indefinitely blocking the service of web application through pop-ups.
Browser Exploitation	Malicious scripts may redirect the user browser to the attacker's site so that the attacker can take full control of the user's computer and use it to install malicious programs like viruses, Trojan horses, etc. and may get access to the user's sensitive information.
Remote Control on System	Once XSS attack vector gets executed on the victim's machine, it will open a way for the attacker to inject different malwares that help in gaining remote access to the victim's system. Thereafter, the system may perform malicious activity on the internet or become a part of the network to launch different attacks such as botnet army.
Phishing [1, 19]	When user clicks on the malicious link sent by the attacker it may redirect the user to the fake web site designed by the attacker to gain access to the sensitive information like the user's login credentials.

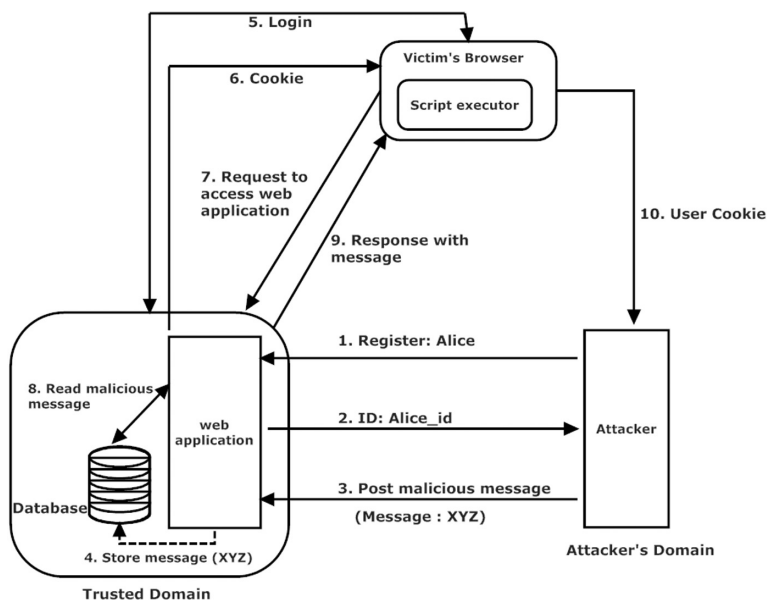


FIGURE 3.1 Persistent XSS attack.

code into the server just once and then affects a large number of benign users with improper sanitization mechanisms. Figure 3.1 depicts the scenario of persistent XSS attack.

3.3.2 Non-Persistent Attack

It is also known as reflected XSS attack as the malicious script gets reflected back in the response by the server. In this attack, the attacker crafts malicious URL link and sends it to the victim using email or posts a fascinating message on social media. When the user clicks on this link a request is sent to the server, but as the request contains script that is not stored on the server, it reflects back the malicious script in response to the user. Now, at the browser side, this script gets executed and the user gets infected by the XSS attack. Figure 3.2 depicts the entire scenario of this attack.

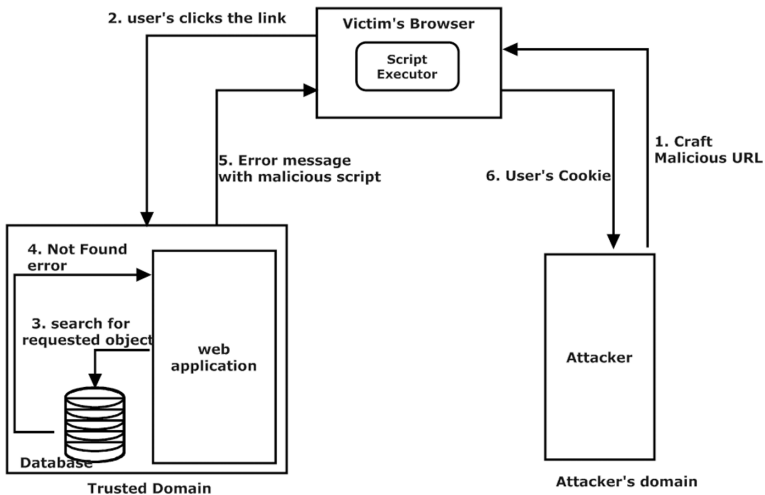


FIGURE 3.2 Non-persistent XSS attack.

3.3.3 DOM-Based XSS Attack

Document Object Model (DOM)-based XSS attack is a client-side XSS attack. DOM enables the browser to process the web content represented by the web page. In this, the injected script is able to alter the structure of the DOM. If it is not properly filtered then it leads to the leakage of the sensitive information. DOM properties like `document.location`, `document.write`, and `document.anchors` may be used by the attacker to launch the XSS attack because these properties are used to access and modify the HTML objects of the web page. This attack is less explored by researchers as it is very hard to detect and mitigate this attack. It requires a careful analysis of the DOM tree while interpreting the web page or response rendered by the server (Figure 3.3).

3.4 APPROACHES TO DEFEND AGAINST XSS ATTACK

It has been discovered by different security organizations that XSS is prevalent in the history of internet security attacks. It has been

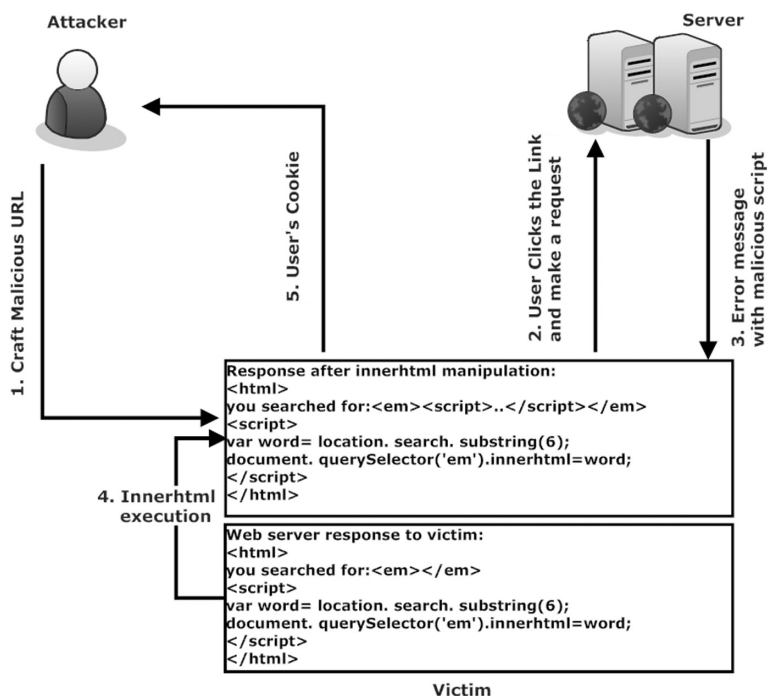


FIGURE 3.3 DOM-based XSS attack.

discovered in almost 80 percent of the web applications including the popular applications like MySpace, Cisco, NASA, Facebook, Twitter, and so many. Therefore, it attracts the attention of different researchers and security solution developers. Different solutions have been designed on the basis of the type of XSS attacks these solutions are dealing with. We have categorized these solutions or approaches into four categories depending on the location of their implementation: client-side approaches, server-side approaches, client-server side approaches, and proxy-based approaches. We will highlight only the major and most effective approaches in the following subsections.

TABLE 3.3 Client-Side Defensive Approaches against XSS Attack

Defensive Approaches	Methodology	Limitations
MLPXSS [24]	Authors have proposed a technique based on ANN-Multilayer Perceptron combined with the dynamic extraction of features for XSS mitigation. This technique performs better when compared to other machine-learning approaches.	It is not tested on real-world web applications for detecting XSS attack.
TT-XSS [39]	Authors have proposed an approach based on dynamic analysis and taint tracking at the browser end to detect DOM-based XSS attack. Here, vulnerabilities are detected through analyzing the flow of suspicious script code execution.	This technique consumes more time to create attack vectors when payloads are complicated and cannot deal with two order inputs.
Khan et al. [21]	Authors have designed an approach that works as interceptor between the client and the server to process the web page to detect malicious code injection in the web page. This technique divides the web page into static and dynamic. Dynamic web pages are tested for any vulnerability by injecting attack payload. If the content is displayed on the page then it is prone to XSS attack.	This technique cannot detect DOM-based XSS attack.
Wang et al. [37]	This technique combines machine-learning classifiers with improved n-gram approach to mitigate XSS attack on the social networking platform.	Training task is challenging because if features and instances are not sufficient, then it may not detect malicious pages.
Guo et al. [7]	Authors have designed an optimized XSS attack vector repository that can be used in detecting XSS attack by the detection tool. Mutation rules are applied on initially constructed XSS attack vector dataset to make it optimized.	It incurs performance overhead while creating optimized XSS repository.

(Continued)

TABLE 3.3 (CONTINUED) Client-Side Defensive Approaches against XSS Attack

Defensive Approaches	Methodology	Limitations
ETSSDetector [29]	This technique works by simulating the behavior of the browser. It interacts with the web page and identifies the suspicious location; then it tests its security by injecting testing payload. If it executes then it is vulnerable to XSS attack.	It is not capable to detect DOM-based XSS attack.
Vishnu et al. [36]	The designed method is based on machine-learning classifiers. Firstly, the dataset is prepared through extracting and analyzing the URL parameter value and JavaScript value, and then it is used for the training of the classifiers to detect XSS attack.	Prepared dataset is not updated automatically. Therefore, it may bypass new attacking payload.
Wang et al. [38]	This technique is based on machine-learning approach that uses ADTree and AdaBoost classifiers to detect XSS attack on social networking sites.	It cannot handle DOM-based XSS attack.
Flashover [35]	This approach is designed to mitigate XSS attack in Adobe Flash-based applications. This approach also depends on static analysis for the identification of suspicious input field and dynamic analysis to test these fields. If testing payload gets executed then it is vulnerable to XSS attack.	Static analysis is effective only in the detection of limited XSS vulnerability sources. And it works only for JavaScript malicious code.
Lekies et al. [23]	Authors have presented an approach that will help in detecting DOM-based XSS attack by using dynamic taint tracking and context-sensitive sanitization.	This technique is not effective against the stored XSS attack.

TABLE 3.4 Server-Side Defensive Approaches against XSS Attack

Defensive Approaches	Methodology	Limitations
Gupta et al. [16]	Authors have developed an approach that relies on finding the mismatch between inserted values and already-known values. It extracts JS code and checks for any deviation from an already-known value for that location. This helps in detecting code injection vulnerabilities like XSS.	Only Javascript context is taken into account, but XSS may also use other contexts like URL parameters and style sheet features. This approach is ineffective against these attack vectors.
DjangoChecker [32]	Authors have designed a dynamic taint analysis tool named as DjangoChecker. This approach effectively identifies whether the sanitizers' primitives that are already applied in the web application are correct at their place. It also identifies the context of attributes where these are applied and examines the correctness of implementing sanitization as per the context. So basically it checks whether sanitization is context sensitive or not.	It is restricted to web application developed using Django and not able to detect DOM-based XSS attack.
Lalia et al. [22]	Authors have proposed an approach to detect malicious script injection by using script features. Here, script features are extracted and then analyzed as to how these are used for crafting malicious scripts. Then, the difference between suspicious script and benign script is identified and used in detecting XSS attack.	This technique is not effective against partial script injection and obfuscated script injection.

(Continued)

TABLE 3.4 (CONTINUED) Server-Side Defensive Approaches against XSS Attack

Defensive Approaches	Methodology	Limitations
Moniruzzaman et al. [25]	Authors have designed a technique that helps in differentiating the actual content of the web page and injected data. This technique is developed only for banking web sites and is based on machine-learning methods. Here, features of DOM tree are extracted and used to train the model.	This technique consumes more time due to feature extraction and sending the web page back to the server.
KameleonFuzz [6]	It is a black-box-detection-based technique which uses fuzz testing for the automatic injection of malicious payload into the web application to activate the XSS vulnerabilities. It basically extends LigRE model two steps further: first is the generation of malicious input, and second is the taint analysis for the vulnerability detection. It protects against stored and reflected XSS attack.	It requires resetting the application which is not practical for live applications. And it requires human interpretation for attack vector generation.
XSSDM [11]	Authors designed an approach that is based on static analysis and pattern matching with context-sensitive sanitization to protect against XSS attack.	This technique requires the manual placing of sanitized code in a web page.
Dong et al. [5]	This approach is basically designed for the webmail system and also possesses the capability to detect XSS attack vectors that are built using new HTML5 features. Here, attack vectors are injected at five injection points in the webmail system, for testing purpose. Then, it is checked whether attack vectors are sanitized correctly or not.	It focuses only on the attack vectors related to the new tags and attribute of the HTML5 and does not take into account other suspicious contexts.

(Continued)

TABLE 3.4 (CONTINUED) Server-Side Defensive Approaches against XSS Attack

Defensive Approaches	Methodology	Limitations
Ruse et al. [30]	This technique is designed for JSP-based web applications and is a concolic testing. It utilizes static analysis with runtime monitoring. It helps in identifying the relationship between input variables and output variables that pave the way for the attacker to initiate XSS attack.	It uses jCute concolic testing which fails to discover test cases in which output variables may have a length of three characters or more.

3.4.1 Client-Side Approaches

These are the approaches, add-ons, or browser extensions that work at the client side. It means these approaches get implemented at the user's machine. We have presented some of the effective approaches as shown in Table 3.3.

3.4.2 Server-Side Approaches

These approaches execute at the server end and defend against XSS attack. Table 3.4 highlights some of the major server-side approaches to defend against XSS attack.

3.4.3 Combinational Approaches

These approaches have both modules to work on the client side as well as server side. Table 3.5 presents the most promising approaches of this category.

3.4.4 Proxy-Based Approaches

These approaches basically act as the proxy between the browser and the server to defend against XSS attack. Some of these approaches are highlighted in Table 3.6.

TABLE 3.5 Combinational Defensive Approaches against XSS Attack

Defensive Approaches	Methodology	Limitations
Gupat et al. [17]	Authors have proposed client-server-based approach that works by separating the JavaScript code into an external file and then analyzing it at the client side. In this technique, suspicious variable context is determined and the decoding of JS is done and finally matches with the injected values in the request. If any match is found then it may indicate XSS attack.	This technique cannot detect DOM-based XSS attack as matching is performed between requesting parameters and response parameters, and DOM-based XSS is client-side vulnerability.
Chaudhary et al. [2]	Authors have proposed a context-sensitive sanitization-based technique. In this approach, the context is determined statically at the server side and dynamically at the client side. After this, sanitizers' primitives are applied as per the context of the vulnerable variable.	This approach does not provide protection against the untrusted script code available from an external source.
Panja et al. [27]	Authors have proposed a technique named as Buffer Based Cache Check to prevent and detect XSS attack on the mobile browser. Cache usage prevents the overhead of providing script whitelist to the web page, again and again. Rather, the server stores verified scripts corresponding to the web page when visited last time. So, if any deviation is found, then it indicates suspicious activity like XSS. It saves time.	This technique requires client- and server-side code modification which incurs performance overhead.

(Continued)

TABLE 3.5 (CONTINUED) Combinational Defensive Approaches against XSS Attack

Defensive Approaches	Methodology	Limitations
Gupat et al. [18]	Authors have proposed an approach to defend against DOM-based XSS attack. Initially, the DOM tree is constructed under normal conditions and scripting nodes are extracted and whitelist is prepared. After this, the DOM tree is constructed for untrusted web pages and extracts the injected script code at identified nodes in the DOM tree. Matching is performed with the whitelist, and any mismatch indicates XSS attack.	This approach may hinder the execution of benign JavaScript code if it does not match with the whitelist.
Nadji et al. [26]	It is based on client-server architecture to enforce document structure integrity. It combines runtime tracking and randomization to thwart XSS attack. This technique ensures integrity constraint, i.e. document structure integrity to prevent malicious data for altering the web application content.	It is not effective to detect the DOM-based XSS attack and requires modification at the client side and server side.

3.5 CHAPTER SUMMARY

As the internet is growing exponentially, it has intertwined into the daily lives of the users as the virtual place where they get faster services of any kind, anywhere, and at any time. It has been adopted by every organization across the globe with the aim of expanding its business. Such proliferation and usability brings

TABLE 3.6 Proxy-Based Defensive Approaches against XSS Attack

Defensive Approaches	Methodology	Limitations
DEXTERJS [28]	It is a robust technique that effectively eliminates the DOM-based XSS attack. It is based on taint tracking and reporting exploit to the client. Basically, it extracts the untrusted JavaScript code and then tests it separately to check if any infection takes place or not by tracking the flow of its execution. Depending upon the information from logs, it generates test payload to verify XSS vulnerability. Once all vulnerabilities are identified, then the exploits are reported to the client.	It incurs performance overhead and does not provide protection against the non-scripting code.
Stock et. al. [33]	This approach basically focuses on identifying the characteristics of suspicious JavaScript code. It utilizes taint tracking browsing system. Firstly, response web page is stored in cache storage and then HTML content is separated from JavaScript code. After this, JS code is examined with the system and some set of metrics are designed that helps in measuring the effect of each attacking flow.	Each type of suspicious flow cannot be detected by this method, for instance, flows that depend on some stated conditions like URL parameter value.
Xiao et. al. [40]	This approach uses the dynamic analysis of JavaScript code embedded in the web page. This technique builds JS abstract syntax tree for internally representing the JavaScript code. Then this tree is forwarded to taint engine that examines this JS code to check whether it attempts to gain access to the sensitive information or not.	It incurs performance overhead and requires lots of computational time to perform its functionalities.

(Continued)

TABLE 3.6 (CONTINUED) Proxy-Based Defensive Approaches against XSS Attack

Defensive Approaches	Methodology	Limitations
Scholte et. al. [31]	Authors have proposed an input validation technique named as IPAAS. Initially, it interrupts the response web page and fetches all the parameters; then it identifies the context of these parameters. This results in the generation of input validation policies, and finally each response web page is examined against these policies. If conditions are not satisfied, then the request is rejected; otherwise it is not.	Type learning can fail in the presence of custom query string formats. In this case, the IPAAS parameter extractor might not be able to reliably parse parameter key-value pairs.
Zhang et. al. [42]	It examines the implementation flow of AJAX application to detect XSS attack. Initially, at the browser side, it checks JavaScript code to design finite state machine for the normal flow of application. Then, this machine is embedded into proxy to monitor the execution flow of each injected script in the response web page. If the execution flow does not match with the finite machine, then it means suspicious flow and may initiate XSS attack.	This technique is not effective against DOM-based XSS attack.

several security issues. One of the major serious concerns is XSS. Therefore, the chapter has focused on elaborating the fundamentals of XSS attack in a very compact and precise manner. We have presented XSS categories with their effects and also provided information related to the defensive approaches developed by the researchers. Last but not the least, XSS cannot go away unless and until the internet users are self-aware about their security and privacy and software developers develop secure software.

REFERENCES

1. Almomani, A., Gupta, B. B., Wan, T. C., Altaher, A., & Manickam, S. (2013) Phishing dynamic evolving neural fuzzy framework for online detection zero-day phishing email. *arXiv preprint arXiv:1302.0629*.
2. Chaudhary, P., Gupta, B. B., & Gupta, S. (2018). Defending the OSN-based web applications from XSS attacks using dynamic javascript code and content isolation. In *Quality, IT and Business Operations* (pp. 107–119). Springer, Singapore.
3. Chaudhary, P., Gupta, S., & Gupta, B. B. (2016). Auditing defense against XSS worms in online social network-based web applications. In *Handbook of Research on Modern Cryptographic Solutions for Computer and Cyber Security* (pp. 216–245). IGI Global.
4. Cross site scripting, OWASP. [online] Available at: [https://www.owasp.org/index.php/Cross-site_Scripting_\(XSS\)](https://www.owasp.org/index.php/Cross-site_Scripting_(XSS)).
5. Dong, G., Zhang, Y., Wang, X., Wang, P., & Liu, L. (2014). Detecting cross site scripting vulnerabilities introduced by HTML5. In *2014 11th International Joint Conference on Computer Science and Software Engineering (JCSSE)*. IEEE.
6. Duchene, F., Rawat, S., Richier, J.-L., & Groz, R. (2014). KameleonFuzz: Evolutionary fuzzing for black-box XSS detection. In *Proceedings of the 4th ACM Conference on Data and Application Security and Privacy* (pp. 37–48). ACM.
7. Guo, X., Jin, S., & Zhang, Y. (2015). XSS vulnerability detection using optimized attack vector repository. In *2015 International Conference On Cyber-Enabled Distributed Computing and Knowledge Discovery (CyberC)*. IEEE.
8. Gupta, B. B. (ed.). (2018). *Computer and Cyber Security: Principles, Algorithm, Applications, and Perspectives*. CRC Press.
9. Gupta, B. B., & Badve, O. P. (2017). Taxonomy of DoS and DDoS attacks and desirable defense mechanism in a cloud computing environment. *Neural Computing and Applications*, 28(12), 3655–3682.
10. Gupta, B. B., Gupta, S., & Chaudhary, P. (2017). Enhancing the browser-side context-aware sanitization of suspicious HTML5 code for halting the DOM-based XSS vulnerabilities in cloud. *International Journal of Cloud Applications and Computing*, 7(1), 1–31.
11. Gupta, M. K., Govil, M. C., Singh, G., & Sharma, P. (2015). XSSDM: Towards detection and mitigation of cross-site scripting vulnerabilities in web applications. In *2015 International Conference*

- on *Advances in Computing, Communications and Informatics (ICACCI)*. IEEE.
12. Gupta, S., & Gupta, B. B. (2015). BDS: Browser dependent XSS sanitizer. In *Handbook of Research on Securing Cloud-Based Databases with Biometric Applications* (pp. 174–191). IGI Global.
 13. Gupta, S., & Gupta, B. B. (2015, May). PHP-sensor: A prototype method to discover workflow violation and XSS vulnerabilities in PHP web applications. In *Proceedings of the 12th ACM International Conference on Computing Frontiers* (p. 59). ACM.
 14. Gupta, S., & Gupta, B. B. (2016). JS-SAN: Defense mechanism for HTML5-based web applications against JavaScript code injection vulnerabilities. *Security and Communication Networks*, 9(11), 1477–1495.
 15. Gupta, S., & Gupta, B. B. (2017). Cross-site scripting (XSS) attacks and defense mechanisms: Classification and state-of-the-art. *International Journal of System Assurance Engineering and Management*, 8(1), 512–530.
 16. Gupta, S., & Gupta, B. B. (2018). A robust server-side javascript feature injection-based design for JSP web applications against XSS vulnerabilities. In *Cyber Security* (pp. 459–465). Springer, Singapore.
 17. Gupta, S., Gupta, B. B., & Chaudhary, P. (2018). A client-server JavaScript code rewriting-based framework to detect the XSS worms from online social network. *Concurrency and Computation: Practice and Experience*, 31(21), e4646.
 18. Gupta, S., Gupta, B. B., & Chaudhary, P. (2018). Hunting for DOM-based XSS vulnerabilities in mobile cloud-based online social network. *Future Generation Computer Systems*, 79, 319–336.
 19. Jain, A. K., & Gupta, B. B. (2017). Phishing detection: Analysis of visual similarity based approaches. *Security and Communication Networks*, 2017.
 20. Jiang, F., Fu, Y., Gupta, B. B., Lou, F., Rho, S., Meng, F., & Tian, Z. (2018). Deep learning based multi-channel intelligent attack detection for data security. *IEEE Transactions on Sustainable Computing*.
 21. Khan, N., Abdullah, J., & Khan, A. S. (2015). Towards vulnerability prevention model for web browser using interceptor approach. In *2015 9th International Conference on IT in Asia (CITA)*. IEEE.
 22. Lalia, S., & Sarah, A. (2018, March). XSS attack detection approach based on scripts features analysis. In *World Conference on Information Systems and Technologies* (pp. 197–207). Springer, Cham.

23. Lekies, S., Stock, B., & Johns, M. (2013). 25 million flows later: Large-scale detection of DOM-based XSS. In *Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security*. ACM.
24. Mokbal, F. M. M., Dan, W., Imran, A., Jiuchuan, L., Akhtar, F., & Xiaoxi, W. (2019). MLPXSS: An integrated XSS-based attack detection scheme in web applications using multilayer perceptron technique. *IEEE Access*, 7, 100567–100580.
25. Moniruzzaman, M., Bagirov, A., Gondal, I., & Brown, S. (2018, June). A server side solution for detecting WebInject: A machine learning approach. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining* (pp. 162–167). Springer, Cham.
26. Nadji, Y., Saxena, P., & Song, D. (2009, February). Document structure integrity: A robust basis for cross-site scripting defense. In *NDSS*.
27. Panja, B., Gennarelli, T., & Meharia, P. (2015). Handling cross site scripting attacks using cache check to reduce webpage rendering time with elimination of sanitization and filtering in light weight mobile web browser. In *2015 First Conference on Mobile and Secure Services (MOBISECserv)*. IEEE.
28. Parameshwaran, E. B., Shinde, S., Dang, H., Sadhu, A., & Saxena, P. (2015). DexterJS: Robust testing platform for DOM-based XSS vulnerabilities. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2015)* (pp. 946–949). ACM.
29. Rocha, T. S., & Souto, E. (2014). ETSSDetector: A tool to automatically detect cross-site scripting vulnerabilities. In *2014 IEEE 13th International Symposium on Network Computing and Applications (NCA)*. IEEE.
30. Ruse, M. E., & Basu, S. (2013). Detecting cross-site scripting vulnerability using concolic testing. In *2013 Tenth International Conference on Information Technology: New Generations (ITNG)*. IEEE.
31. Scholte, T., Robertson, W., Balzarotti, D., & Kirda, E. (2012). Preventing input validation vulnerabilities in web applications through automated type analysis. In *2012 IEEE 36th Annual Computer Software and Applications Conference (COMPSAC)*. IEEE.
32. Steinhauser, A., & Tüma, P. (2019). DjangoChecker: Applying extended taint tracking and server side parsing for detection of context-sensitive XSS flaws. *Software: Practice and Experience*, 49(1), 130–148.

33. Stock, B., Pfister, S., Kaiser, B., Lekies, S., & Johns, M. (2015). From facepalm to brain bender: Exploring client-side cross-site scripting. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS '15)* (pp. 1419–1430). ACM.
34. Tripathi, S., Gupta, B., Almomani, A., Mishra, A., & Veluru, S. (2013). Hadoop based defense solution to handle distributed denial of service (ddos) attacks. *Journal of Information Security*, 04(3), 150.
35. Van Acker, S., Nikiforakis, N., Desmet, L., Joosen, W., & Piessens, F. (2012). FlashOver: Automated discovery of cross-site scripting vulnerabilities in rich internet applications. In *Proceedings of the 7th ACM Symposium on Information, Computer and Communications Security* (pp. 12–13). ACM.
36. Vishnu, B. A., & Jevitha, K. P. (2014). Prediction of cross-site scripting attack using machine learning algorithms. In *Proceedings of the 2014 International Conference on Interdisciplinary Advances in Applied Computing (ICONIAAC '14)*. ACM.
37. Wang, R., Jia, X., Li, Q., & Zhang, D. (2015). Improved N-gram approach for cross-site scripting detection in Online Social Network. In *2015 Science and Information Conference (SAI)*. IEEE.
38. Wang, R., Jia, X., Li, Q., & Zhang, S. (2014). Machine learning based cross-site scripting detection in online social network. In *2014 IEEE International Conference on High Performance Computing and Communications, 2014 IEEE 6th International Symposium on Cyberspace Safety and Security, 2014 IEEE 11th International Conference on Embedded Software and Syst (HPCC, CSS, ICESS)*. IEEE.
39. Wang, R., Xu, G., Zeng, X., Li, X., & Feng, Z. (2018). TT-XSS: A novel taint tracking based dynamic detection framework for DOM cross-site scripting. *Journal of Parallel and Distributed Computing*, 118, 100–106.
40. Xiao, W., Sun, J., Chen, H., & Xu, X. (2014). Preventing client side XSS with rewrite based dynamic information flow. In *2014 Sixth International Symposium on Parallel Architectures, Algorithms and Programming (PAAP)*. IEEE.
41. XSS incidents information. [online] Available at: <http://www.xssed.com/>.
42. Zhang, Q., Chen, H., & Sun, J. (2010). An execution-flow based method for detecting cross-site scripting attacks. In *2010 2nd International Conference on Software Engineering and Data Mining (SEDM)*. IEEE.